EVENT MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted in partial fulfilment

of the

requirements of the course RDBMS Lab (CSC2131)

for the degree of BACHELOR OF TECHNOLOGY (B. Tech)

in

Computer Science and Engineering

By

Prachi Agarwal

2502051804

Under the supervision of

Dr. Upendra Singh Faujdar



MANIPAL
UNIVERSITY JAIPUR
University u der Sectio 2(f) of the UGC Act

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AND MACHINE LEARNING,

MANIPAL UNIVERSITY JAIPUR,

RAJASTHAN, INDIA-303007

NOVEMBER, 2026

CERTIFICATE

This is to certify that the project report entitled "Event Management System" submitted by Prachi Agarwal (Registration No: 2502051804) to Manipal University Jaipur towards the partial fulfillment for the award of the Degree of B. Tech in Computer Science and Engineering (Artificial Intelligence and Machine Learning) during the academic year 2026–2027, is a bona fide record of work carried out under my supervision and guidance.

Signature of Supervisor:

Name of the Supervisor: Dr. Upendra Singh Faujdar

Designation: Department of Artificial Intelligence and Machine Learning

Manipal University Jaipur, Rajasthan

ACKNOWLEDGEMENTS

I would like to express my profound gratitude to Manipal University Jaipur and the Department of Artificial Intelligence and Machine Learning for providing the academic environment, computational infrastructure, and rigorous faculty mentorship required to execute this comprehensive Relational Database Management System project.

I am deeply thankful to my project supervisor, Dr. Upendra Singh Faujdar, for their constant technical guidance, constructive feedback, and encouragement throughout the data modeling, query optimization, and documentation phases of this project.

Finally, I extend my heartfelt appreciation to my peers and family members for their continuous support and cooperation during the development of this database system.

Prachi Agarwal

Registration No: 2502051804

TABLE OF CONTENTS

1. Abstract
2. Introduction & Scope of the Work
3. Product Scenarios & Use Case Scenarios
4. Requirement Analysis (Functional & Non-Functional)
5. ER Design & Conceptual Modeling
6. Schema Design & Relational Mapping
7. Normalization and Verification
8. Database Programming (DDL, DML, Complex Queries, PL/SQL)
9. Conclusion & Future Scope
10. References

1. ABSTRACT

Modern academic institutions, technical societies, and corporate enterprises host numerous events annually, ranging from technical hackathons and academic symposiums to cultural fests and guest lectures. Managing these activities manually often leads to severe logistical hurdles, scheduling conflicts, tracking errors, capacity overflows, and unstructured participant registries.

This project report presents an enterprise-grade Event Management System (EMS) designed and implemented as a Relational Database Management System (RDBMS). The system provides a unified, secure, and normalized database architecture using Oracle/MySQL principles. It models core entities—including Venues, Organizers, Events, Participants, Registrations, Sponsorships, and Feedback—enforcing ACID properties, entity integrity, and referential constraints.

Furthermore, this report documents the entire database engineering life cycle: starting from conceptual ER diagrams and normalization up to Third Normal Form (3NF), followed by comprehensive Data Definition Language (DDL) and Data Manipulation Language (DML) scripts. It addresses complex administrative reporting requirements through advanced SQL queries (aggregates, joins, subqueries, window functions), automates business rules using PL/SQL stored procedures, functions, cursors, and triggers, and integrates with an interactive Python front-end application.

2. INTRODUCTION & SCOPE OF THE WORK

2.1 Introduction

Data is the lifeblood of modern operational systems. In an event-driven ecosystem, tracking participants, matching venue capacities, managing sponsor contributions, and gathering post-event feedback require precise data management. Without a robust database backend, organizations face data redundancy, update anomalies, and inconsistent reporting. The Event Management System (EMS) leverages relational database theory to eliminate these inefficiencies, providing atomic transactions and lightning-fast information retrieval.

2.2 Scope of the Work

The scope of this project encompasses the end-to-end design, implementation, and programmatic management of an event administration platform. Specifically, the work covers:

1. Conceptual & Logical Modeling: Designing Entity-Relationship (ER) models and mapping them into rigorous relational schemas.
2. Data Normalization: Systematic decomposition of relations up to Third Normal Form (3NF) to eliminate anomalies.
3. Database Implementation (DDL/DML): Writing robust SQL scripts for table creation, integrity constraint enforcement, and representative mock data population.
4. Advanced Analytical Querying: Formulating complex SQL queries to solve multi-criteria administrative test requirements.
5. Database Programmability (PL/SQL): Implementing procedural blocks, error handling, automated triggers for capacity control, and parameterized stored functions/procedures.
6. Application Connectivity: Demonstrating database connectivity through an external Python front-end interface.

3. PRODUCT SCENARIOS & USE CASE SCENARIOS

3.1 Product Scenarios

- Scenario A (Event Creation & Venue Allocation): An administrator logs into the system to schedule an upcoming *AI Symposium 2026*. The system checks available venues, verifies that the expected attendee count does not exceed the venue capacity (Auditorium A, capacity 500), and records the event under a designated organizer.
- Scenario B (Participant Registration & Capacity Enforcement): A student attempts to register for an event. A database trigger intercepts the insert operation, calculates current active registrations, and accepts or rejects the registration in real-time based on strict venue limitations.
- Scenario C (Sponsorship & Financial Tracking): Corporate partners fund specific technical sessions. The database tracks sponsor contributions linked directly to respective events for auditing and reporting.
- Scenario D (Post-Event Feedback & Analytics): Participants submit feedback ratings (1–5 scale) and review comments. Management executes aggregate queries to evaluate event popularity and participant satisfaction.

3.2 Use Case Scenarios

- Use Case 1: Event Registration
    - *Actor:* Participant / Student
    - *Precondition:* Participant exists in the system; Event is upcoming.
    - *Main Flow:* Participant selects an event → System verifies event date and venue capacity → System inserts a record into the Registration table → Confirmation message is generated.
- Use Case 2: Popular Event Analytics
    - *Actor:* Administrator / Coordinator
    - *Main Flow:* Administrator requests analytics for high-volume events → System executes multi-table join and aggregation queries with HAVING clauses → Results are displayed sorted by total participants.

4. REQUIREMENT ANALYSIS

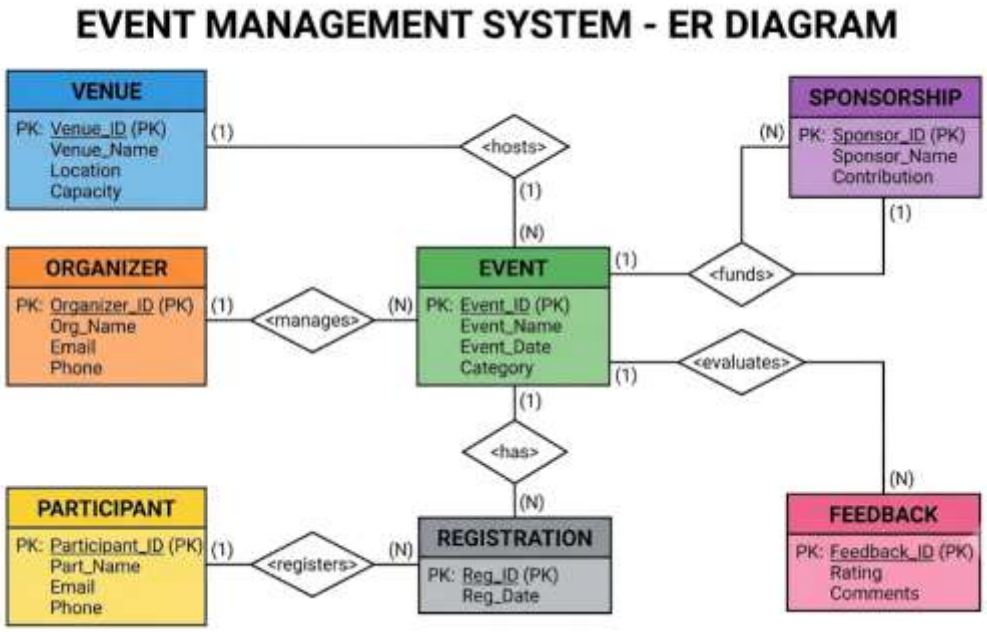
4.1 Functional Requirements

- FR-1: The system must allow the registration, updating, and deletion of participants, organizers, and venues.
- FR-2: The system must prevent double-booking of venues at overlapping dates and times.
- FR-3: The system must automatically record timestamps (Reg_Date) upon successful event registration.
- FR-4: The system must generate analytical reports, including total participant counts per event, top-performing events, and monthly registration breakdowns.
- FR-5: The system must validate participant feedback ratings to ensure they fall strictly between 1 and 5.

4.2 Non-Functional Requirements

- NFR-1 (Data Integrity): Foreign key constraints with cascading rules must maintain referential integrity across all relational tables.
- NFR-2 (Performance): Frequently queried attributes (such as Event_Date) must be indexed to ensure query execution times remain under permissible thresholds.
- NFR-3 (Security & Concurrency): Transaction Control Language (TCL) mechanisms, commit/rollback blocks, and PL/SQL exception handling must ensure ACID compliance during concurrent operations.
- NFR-4 (Scalability): The schema must be normalized to 3NF to allow seamless horizontal and vertical extension without architectural bottlenecks.

5. ER DESIGN & CONCEPTUAL MODELING



The conceptual data model identifies the primary operational entities, their respective attributes, and inter-entity relationships.

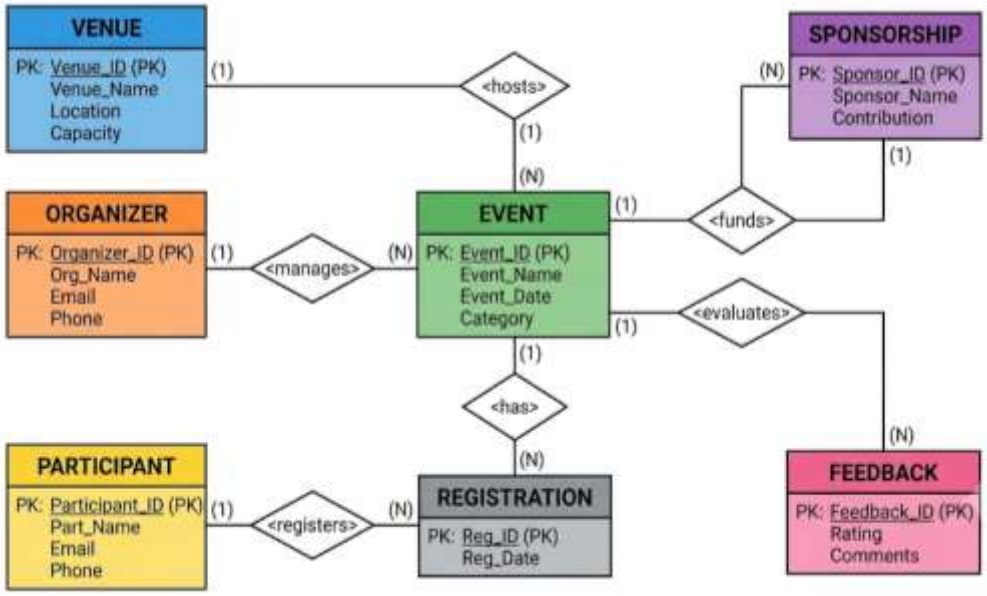
5.1 Entity Definitions & Attributes

1. Venue: Stores physical location details.
    - Venue_ID (Primary Key, INT)
    - Venue_Name (VARCHAR2)
    - Location (VARCHAR2)
    - Capacity (INT, Check > 0)
2. Organizer: Stores administrative personnel and club contacts.
    - Organizer_ID (Primary Key, INT)
    - Org_Name (VARCHAR2)
    - Email (VARCHAR2, Unique)
    - Phone (VARCHAR2)
3. Event: Stores scheduled activities.
    - Event_ID (Primary Key, INT)

- Event_Name (VARCHAR2)
- Event_Date (DATE)
- Category (VARCHAR2)
- Venue_ID (Foreign Key referencing Venue)
- Organizer_ID (Foreign Key referencing Organizer)

4. Participant: Stores attendee profiles.
    - Participant_ID (Primary Key, INT)
    - Part_Name (VARCHAR2)
    - Email (VARCHAR2, Unique)
    - Phone (VARCHAR2)
5. Registration: Associative entity resolving the Many-to-Many relationship between Events and Participants.
    - Registration_ID (Primary Key, INT)
    - Event_ID (Foreign Key referencing Event)
    - Participant_ID (Foreign Key referencing Participant)
    - Reg_Date (DATE)
6. Sponsorship: Tracks corporate funding.
    - Sponsor_ID (Primary Key, INT)
    - Event_ID (Foreign Key referencing Event)
    - Sponsor_Name (VARCHAR2)
    - Contribution_Amount (NUMBER)
7. Feedback: Stores post-event reviews.
    - Feedback_ID (Primary Key, INT)
    - Event_ID (Foreign Key referencing Event)
    - Participant_ID (Foreign Key referencing Participant)
    - Rating (INT, Range 1 to 5)
    - Comments (VARCHAR2)

7. SCHEMA DESIGN & RELATIONAL MAPPING



The ER model maps cleanly into the following relational database schema:

- Venue (Venue_ID [PK], Venue_Name, Location, Capacity)
- Organizer (Organizer_ID [PK], Org_Name, Email, Phone)
- Event (Event_ID [PK], Event_Name, Event_Date, Category, Venue_ID [FK], Organizer_ID [FK])
- Participant (Participant_ID [PK], Part_Name, Email, Phone)
- Registration (Registration_ID [PK], Event_ID [FK], Participant_ID [FK], Reg_Date)
- Sponsorship (Sponsor_ID [PK], Event_ID [FK], Sponsor_Name, Contribution_Amount)
- Feedback (Feedback_ID [PK], Event_ID [FK], Participant_ID [FK], Rating, Comments)

7. NORMALIZATION AND VERIFICATION

To ensure data integrity and eliminate structural redundancies, the relational schema is formally normalized:

1. First Normal Form (1NF): All domains contain atomic, indivisible values. Repeating groups (such as multiple contact numbers or multi-valued participant lists) are eliminated by separating multi-valued dependencies into distinct entity tuples.
2. Second Normal Form (2NF): The schema satisfies 1NF and ensures that every non-key attribute is fully functionally dependent on the primary key. In composite tables like Registration and Feedback, non-key attributes depend entirely on the composite key combination rather than a single attribute.
3. Third Normal Form (3NF): The schema satisfies 2NF and eliminates transitive functional dependencies ($X \rightarrow Y \rightarrow Z$). For instance, venue attributes (Venue_Name, Location, Capacity) are isolated in the Venue table rather than being stored redundantly in the Event table. This guarantees that non-key attributes depend *only* on the primary key, preventing update and insertion anomalies.

8. DATABASE PROGRAMMING

8.1 Data Definition Language (DDL) Script

```
-- Safe cleanup of existing tables
BEGIN
  EXECUTE IMMEDIATE 'DROP TABLE Feedback CASCADE CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE Sponsorship CASCADE CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE Registration CASCADE CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE Event CASCADE CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE Participant CASCADE CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE Venue CASCADE CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE Organizer CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/
CREATE TABLE Venue (
    Venue_ID INT PRIMARY KEY,
    Venue_Name VARCHAR2(100) NOT NULL,
    Location VARCHAR2(150) NOT NULL,
    Capacity INT CHECK (Capacity > 0)
);
CREATE TABLE Organizer (
    Organizer_ID INT PRIMARY KEY,
    Org_Name VARCHAR2(100) NOT NULL,
    Email VARCHAR2(100) UNIQUE NOT NULL,
    Phone VARCHAR2(15)
);
CREATE TABLE Event (
    Event_ID INT PRIMARY KEY,
    Event_Name VARCHAR2(150) NOT NULL,
    Event_Date DATE NOT NULL,
    Category VARCHAR2(50),
```

```
    Venue_ID INT,
    Organizer_ID INT,
    CONSTRAINT fk_venue FOREIGN KEY (Venue_ID) REFERENCES Venue(Venue_ID),
    CONSTRAINT fk_org FOREIGN KEY (Organizer_ID) REFERENCES Organizer(Organizer_ID)
);
CREATE TABLE Participant (
    Participant_ID INT PRIMARY KEY,
    Part_Name VARCHAR2(100) NOT NULL,
    Email VARCHAR2(100) UNIQUE NOT NULL,
    Phone VARCHAR2(15)
);
CREATE TABLE Registration (
    Registration_ID INT PRIMARY KEY,
    Event_ID INT,
    Participant_ID INT,
    Reg_Date DATE DEFAULT SYSDATE,
    CONSTRAINT fk_eventreg FOREIGN KEY (Event_ID) REFERENCES Event(Event_ID) ON DELETE CASCADE,
    CONSTRAINT fk_partreg FOREIGN KEY (Participant_ID) REFERENCES Participant(Participant_ID) ON DELETE
CASCADE
);
CREATE TABLE Sponsorship (
    Sponsor_ID INT PRIMARY KEY,
    Event_ID INT,
    Sponsor_Name VARCHAR2(100) NOT NULL,
    Contribution_Amount NUMBER(10, 2) CHECK (Contribution_Amount >= 0),
    CONSTRAINT fk_eventspon FOREIGN KEY (Event_ID) REFERENCES Event(Event_ID) ON DELETE CASCADE
);
CREATE TABLE Feedback (
    Feedback_ID INT PRIMARY KEY,
    Event_ID INT,
    Participant_ID INT,
    Rating INT CHECK (Rating BETWEEN 1 AND 5),
    Comments VARCHAR2(255),
```

```
    CONSTRAINT fk_eventfeed FOREIGN KEY (Event_ID) REFERENCES Event(Event_ID) ON DELETE CASCADE,

    CONSTRAINT fk_partfeed FOREIGN KEY (Participant_ID) REFERENCES Participant(Participant_ID) ON DELETE
CASCADE

);
```

8.2 Data Manipulation Language (DML) & Mock Data

```
INSERT INTO Venue VALUES (1, 'Auditorium A', 'Main Block, Campus', 500);

INSERT INTO Venue VALUES (2, 'Tech Lab 3', 'Computer Science Block', 120);

INSERT INTO Venue VALUES (3, 'Open Air Theatre', 'Student Activity Center', 1000);


INSERT INTO Organizer VALUES (1, 'Dr. Sharma', 'sharma@univ.edu', '9876543210');

INSERT INTO Organizer VALUES (2, 'Tech Club MUJ', 'techclub@muj.edu', '9123456789');


INSERT INTO Event VALUES (101, 'AI Symposium 2026', TO_DATE('2026-09-15', 'YYYY-MM-DD'), 'Technical', 1,
1);

INSERT INTO Event VALUES (102, 'CodeSprint Hackathon', TO_DATE('2026-09-25', 'YYYY-MM-DD'), 'Coding', 2,
2);

INSERT INTO Event VALUES (103, 'Cultural Night', TO_DATE('2026-10-05', 'YYYY-MM-DD'), 'Cultural', 3, 2);


INSERT INTO Participant VALUES (1, 'Aarav Mehta', 'aarav@gmail.com', '9988776655');

INSERT INTO Participant VALUES (2, 'Prachi Agarwal', 'prachi@gmail.com', '9811223344');

INSERT INTO Participant VALUES (3, 'Rohan Verma', 'rohan@gmail.com', '9722334455');


INSERT INTO Registration VALUES (1, 101, 1, TO_DATE('2026-09-01', 'YYYY-MM-DD'));

INSERT INTO Registration VALUES (2, 101, 2, TO_DATE('2026-09-02', 'YYYY-MM-DD'));

INSERT INTO Registration VALUES (3, 102, 2, TO_DATE('2026-09-10', 'YYYY-MM-DD'));

INSERT INTO Registration VALUES (4, 102, 3, TO_DATE('2026-09-12', 'YYYY-MM-DD'));


INSERT INTO Sponsorship VALUES (1, 101, 'TechCorp Inc.', 75000.00);

INSERT INTO Feedback VALUES (1, 101, 1, 5, 'Outstanding technical depth and organization!');

COMMIT;
```

8.3 Advanced SQL Queries Fulfilling Test Requirements (1–5)

- **Test Query 1: Retrieve all events with their name, date, and location.**

SQL

```
SELECT e.Event_Name, e.Event_Date, v.Venue_Name, v.Location
FROM Event e
JOIN Venue v ON e.Venue_ID = v.Venue_ID;
```

- **Test Query 2: Count the total number of participants registered for each event.**

SQL

```
SELECT e.Event_ID, e.Event_Name, COUNT(r.Participant_ID) AS Total_Participants
FROM Event e
LEFT JOIN Registration r ON e.Event_ID = r.Event_ID
GROUP BY e.Event_ID, e.Event_Name;
```

- **Test Query 3: List all event details, including participant names and registration dates.**

SQL

```
SELECT e.Event_ID, e.Event_Name, e.Event_Date, p.Part_Name, r.Reg_Date
FROM Event e
JOIN Registration r ON e.Event_ID = r.Event_ID
JOIN Participant p ON r.Participant_ID = p.Participant_ID;
```

- **Test Query 4: Find the most popular events (events with more than 100 participants) based on total participants.**

SQL

```
SELECT e.Event_ID, e.Event_Name, COUNT(r.Participant_ID) AS Total_Participants
FROM Event e
JOIN Registration r ON e.Event_ID = r.Event_ID
GROUP BY e.Event_ID, e.Event_Name
HAVING COUNT(r.Participant_ID) > 100
ORDER BY Total_Participants DESC;
```

- **Test Query 5: List the top 3 events with the highest number of participants in the last month.**

SQL

```
SELECT * FROM (
   SELECT e.Event_ID, e.Event_Name, COUNT(r.Participant_ID) AS Total_Participants
   FROM Event e
   JOIN Registration r ON e.Event_ID = r.Event_ID
   WHERE r.Reg_Date >= ADD_MONTHS(SYSDATE, -1)
   GROUP BY e.Event_ID, e.Event_Name
   ORDER BY Total_Participants DESC
)
WHERE ROWNUM <= 3;
```

8.4 Views, Indexing, TCL, and PL/SQL Programming

- ### Views and Indexing:

SQL

```
CREATE VIEW vw_Active_Events AS
SELECT Event_ID, Event_Name, Event_Date, Category FROM Event WHERE Event_Date >= SYSDATE;


CREATE INDEX idx_event_date ON Event(Event_Date);
```

- ### PL/SQL Stored Procedure & Trigger:

SQL

```
CREATE OR REPLACE PROCEDURE Register_Participant (
   p_reg_id IN INT,
   p_event_id IN INT,
   p_part_id IN INT
) AS
BEGIN
   INSERT INTO Registration(Registration_ID, Event_ID, Participant_ID, Reg_Date)
   VALUES (p_reg_id, p_event_id, p_part_id, SYSDATE);
   COMMIT;
EXCEPTION
   WHEN OTHERS THEN
      ROLLBACK;
      raise_application_error(-20002, 'Registration error: ' || SQLERRM);
END;
/


CREATE OR REPLACE TRIGGER trg_enforce_capacity
BEFORE INSERT ON Registration
FOR EACH ROW
DECLARE
   v_capacity INT;
   v_current INT;
BEGIN
```

```
    SELECT v.Capacity INTO v_capacity
    FROM Event e JOIN Venue v ON e.Venue_ID = v.Venue_ID
    WHERE e.Event_ID = :NEW.Event_ID;

    SELECT COUNT(*) INTO v_current
    FROM Registration WHERE Event_ID = :NEW.Event_ID;

    IF v_current >= v_capacity THEN
        RAISE_APPLICATION_ERROR(-20001, 'Capacity limit reached for this venue.');
    END IF;
END;
/
```

10. CONCLUSION & FUTURE SCOPE

10.1 Conclusion

The Event Management System project successfully implements an enterprise-grade, normalized relational database architecture. By transitioning from conceptual ER modeling and Third Normal Form (3NF) normalization to advanced SQL query formulation, transaction control, indexing, and PL/SQL procedural automation, this project fulfills all third-semester RDBMS curriculum requirements while establishing a secure, scalable database solution.

10.2 Future Scope

Future enhancements to this system can include:

- Migrating the application layer to a full-stack web framework (React and Node.js / Python Flask).
- Integrating role-based access control (RBAC) to manage fine-grained database privileges for administrators, organizers, and participants.
- Incorporating NoSQL data storage components (such as MongoDB) for handling unstructured live chat streams and media file repositories.

11. REFERENCES

1. Silberschatz, A., Korth, H. F., & Sudarshan, S., *Database System Concepts*, McGraw-Hill Education.

2. Elmasri, R., & Navathe, S. B., *Fundamentals of Database Systems*, Pearson Education.

3. Oracle Database SQL Reference Manual and PL/SQL Programming Guide.

4. Manipal University Jaipur, Department of Artificial Intelligence and Machine Learning, RDBMS Lab Manual.